Testing tools¶

Test araçları

Django, test yazarken işinize yarayan küçük bir araç seti sağlar.

The test client¶

Test istemcisi

Test istemcisi, sahte bir web tarayıcısı gibi davranan bir Python sınıfıdır. Bu sayede view'larınızı test edebilir ve Django ile oluşturulmuş uygulamanızla programatik olarak etkileşime geçebilirsiniz.

Test istemcisi ile yapabileceklerinizden bazıları:

- Bir URL üzerinde GET ve POST istekleri simüle edip sonucu gözlemlemek – düşük seviyeli HTTP (header'lar ve durum kodları) dahil olmak üzere sayfa içeriğine kadar her şey.
- Yönlendirme zincirini (varsa) görmek ve her adımda URL ve durum kodunu kontrol etmek.
- Belirli bir isteğin, belirli bir Django template'i ile render edilip edilmediğini ve template context'inde belirli değerlerin olup olmadığını test etmek.

Not: Test istemcisi Selenium gibi "tarayıcı içi" framework'lerin yerine geçmek için tasarlanmamıştır. Django test istemcisinin amacı farklıdır. Kısaca:

- Doğru template'in render edildiğini ve doğru context verisinin gönderildiğini test etmek için Django test istemcisini kullanın.
- View fonksiyonlarını doğrudan test etmek (routing ve middleware'i atlayarak) için RequestFactory kullanın.
- HTML çıktısını ve sayfanın davranışını (özellikle JavaScript) test etmek için Selenium gibi araçlar kullanın.

Kapsamlı bir test paketi bu test türlerinin hepsini birlikte kullanmalıdır.

Overview and a quick example¶

Genel bakış ve hızlı bir örnek

Test istemcisini kullanmak için django.test.Client oluşturup web sayfalarını çekebilirsiniz:

```
>>> from django.test import Client
>>> c = Client()
>>> response = c.post("/login/", {"username": "john", "password":
"smith"})
>>> response.status_code
200
>>> response = c.get("/customer/details/")
>>> response.content
b'<!DOCTYPE html...'
```

Bu örnekten de görüldüğü gibi, Client sınıfını Python interpreter içinde oluşturabilirsiniz.

Test istemcisinin çalışma şekli hakkında önemli noktalar:

- Test istemcisi çalışmak için web server'a ihtiyaç duymaz. Hatta server hiç çalışmıyor olsa bile çalışır! Çünkü HTTP katmanını atlayıp doğrudan Django ile iletişim kurar. Bu da testlerin hızlı çalışmasını sağlar.
- Sayfa çekerken tam domain değil, sadece path kullanılmalıdır:

Doğru:

```
c.get("/login/")
```

Yanlış:

```
c.get("https://www.example.com/login/")
```

- Test istemcisi, Django projeniz dışındaki siteleri çekemez. Bunun için urllib gibi Python modüllerini kullanmalısınız.
- URL çözümleme işlemi ROOT_URLCONF ayarına göre yapılır.
- Bazı özellikler (özellikle template ile ilgili olanlar) sadece test çalışırken aktif olur.
- Bunun nedeni Django'nun test sırasında template sistemini geçici olarak değiştirmesidir.
- Varsayılan olarak CSRF kontrolleri devre dışıdır.
- CSRF kontrolü yapmak için:

```
Client(enforce_csrf_checks=True)
```

Making requests¶

İstek gönderme

django.test.Client sınıfı ile istek gönderilir.

```
class Client(...)
```

Bu sınıf çeşitli parametrelerle özelleştirilebilir.

- headers: Her istekte gönderilecek header'lar
- query_params: Varsayılan query string
- **defaults: WSGI environment değişkenleri

Örnek:

```
client = Client(headers={"user-agent": "curl/7.79.1"})
```

GET isteği

```
get(path, data=None, follow=False, secure=False, ...)
```

GET isteği yapar ve Response döner.

```
c.get("/customers/details/", query_params={"name": "fred", "age":
7})
```

Sonuç:

```
/customers/details/?name=fred&age=7
```

Header ekleme:

```
c.get("/url/", headers={"accept": "application/json"})
```

- follow=True → yönlendirmeleri takip eder
- secure=True → HTTPS gibi davranır

POST isteği

```
post(path, data=None, content_type=..., ...)
```

POST isteği yapar.

```
c.post("/login/", {"name": "fred", "passwd": "secret"})
```

Gönderilen veri:

```
name=fred&passwd=secret
```

- JSON gönderimi için content_type="application/json"
- Dosya yüklemek için:

```
with open("file.txt", "rb") as f:
    c.post("/upload/", {"file": f})
```

Diğer HTTP metodları

- head()
- options()
- put()
- patch()
- delete()
- trace()

Hepsi benzer şekilde çalışır.

Login işlemleri

```
c.login(username="fred", password="secret")
```

- Başarılıysa True döner
- Test ortamında kullanıcı oluşturmanız gerekir

Alternatif:

```
c.force_login(user)
```

- Şifre kontrolü yapmaz (daha hızlı)

Çıkış:

```
c.logout()
```

Testing responses¶

Response test etme

Response objesi şu özelliklere sahiptir:

- content → sayfa içeriği (bytes)
- context → template verisi
- status_code → HTTP durum kodu
- json() → JSON parse eder
- templates → kullanılan template'ler
- resolver_match → hangi view çalıştı

Örnek:

```
response.context["name"]
```

Exceptions¶

Hatalar

- Hata oluşursa exception fırlatılır
- Bazı hatalar (404 vs.) HTTP response'a çevrilir
- raise_request_exception=False ise 500 döner

Persistent state¶

Kalıcı durum

Client stateful'dur:

- Cookie'leri saklar
- Session bilgisi tutar

```
session = self.client.session
session["key"] = "value"
session.save()
```

Setting the language¶

Dil ayarlama

Cookie ile:

```
self.client.cookies.load({"django_language": "fr"})
```

Header ile:

```
headers={"accept-language": "fr"}
```

Example¶

Örnek

```
import unittest
from django.test import Client

class SimpleTest(unittest.TestCase):
    def setUp(self):
        self.client = Client()

    def test_details(self):
        response = self.client.get("/customer/details/")

        self.assertEqual(response.status_code, 200)
        self.assertEqual(len(response.context["customers"]), 5)
```

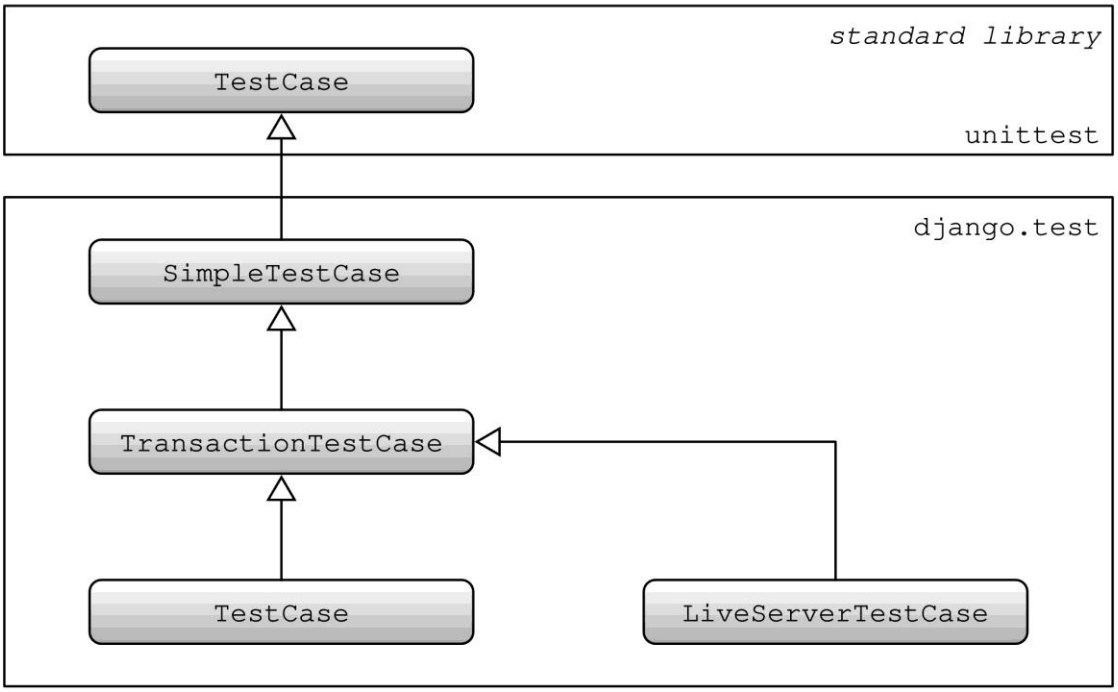
Ayrıca bakınız

django.test.RequestFactory

Provided test case classes¶

Sağlanan test sınıfları

Normal Python unit test sınıfları, unittest.TestCase adlı temel bir sınıftan türetilir. Django bu temel sınıfın birkaç genişletilmiş versiyonunu sağlar:



Hierarchy of Django unit testing classes¶

Django test sınıflarının hiyerarşisi

Normal bir unittest.TestCase sınıfını, Django'nun alt sınıflarından birine dönüştürebilirsiniz: bunun için sadece base class'ı değiştirmeniz yeterlidir.

Standart Python test özelliklerinin tamamı kullanılabilir olmaya devam eder ve aşağıda açıklanan ek özelliklerle genişletilir.

SimpleTestCase¶

`class SimpleTestCase`

unittest.TestCase'in bir alt sınıfıdır ve şu özellikleri ekler:

- Faydalı assertion'lar:
  - Bir callable'ın belirli bir exception fırlattığını kontrol etmek
  - Bir callable'ın belirli bir warning oluşturduğunu kontrol etmek
  - Form field render ve hata kontrolü
  - HTML response içinde belirli bir parçanın olup olmadığını test etmek
  - Bir template'in kullanılıp kullanılmadığını doğrulamak
  - İki URL'nin eşitliğini kontrol etmek

    - HTTP redirect olup olmadığını kontrol etmek
    - HTML, XML ve JSON karşılaştırmalarını sağlam şekilde yapmak
- Ayarları değiştirerek test çalıştırabilme
- Client kullanımı

Eğer testleriniz veritabanı kullanıyorsa TransactionTestCase veya TestCase kullanın.

SimpleTestCase.databases

- Varsayılan olarak veritabanı sorgularına izin vermez.
- Bu, testler arasında veri karışmasını önlemek içindir.
- Eğer izin vermek isterseniz:

```
databases = "__all__"
```

Uyarı

SimpleTestCase ve alt sınıfları setUpClass() ve tearDownClass() kullanır.

Override ederseniz:

```
class MyTestCase(TestCase):
    @classmethod
    def setUpClass(cls):
        super().setUpClass()

    @classmethod
    def tearDownClass(cls):
        super().tearDownClass()
```

- setUpClass() içinde hata olursa testler çalışmaz
- super() çağrılmazsa transaction sızıntısı olabilir

TransactionTestCase¶

```
class TransactionTestCase
```

SimpleTestCase'den türetilmiştir ve veritabanı özellikleri ekler:

- Test sonunda veritabanını sıfırlar
- Fixture desteği

- DB özelliklerine göre test atlama
- Ek assertion'lar

TestCase ile farkı

- TransactionTestCase:
  - Tüm tabloları truncate eder
  - commit ve rollback test edilebilir
- TestCase:
  - Transaction kullanır
  - Test sonunda rollback yapılır

Uyarı

Rollback desteklemeyen DB'lerde (örneğin MyISAM):

- Tüm veri silinerek sıfırlama yapılır

Ek olarak:

```
serialized_rollback = True
```

kullanılabilir.

TestCase¶

```
class TestCase
```

En yaygın kullanılan sınıftır.

Özellikleri:

- TransactionTestCase'den türetilir
- Testleri atomic blok içinde çalıştırır
- DB constraint kontrolü yapar

setUpTestData()

```
@classmethod
def setUpTestData(cls):
```

- Tüm testler için ortak veri oluşturur
- setUp()'a göre daha hızlıdır

captureOnCommitCallbacks()

Transaction callback'lerini yakalar:

```
with self.captureOnCommitCallbacks(execute=True) as callbacks:
```

LiveServerTestCase¶

```
class LiveServerTestCase
```

Ek özellik:

- Arka planda gerçek bir Django server başlatır
- Selenium gibi araçlarla gerçek kullanıcı simülasyonu yapılır

Server adresi:

```
self.live_server_url
```

Selenium örneği

```
from selenium.webdriver.firefox.webdriver import WebDriver

class MySeleniumTests(StaticLiveServerTestCase):
    @classmethod
    def setUpClass(cls):
        cls.selenium = WebDriver()

    def test_login(self):
        self.selenium.get(f"{self.live_server_url}/login/")
```

Önemli not

SQLite memory DB kullanırken:

- Aynı bağlantı iki thread tarafından paylaşılır
- Aynı anda sorgu yapılmamalıdır

Test cases features¶

Test özellikleri

Default test client

Her testte:

```
self.client
```

kullanılabilir.

Custom client

```
class MyTest(TestCase):
    client_class = MyTestClient
```

Fixture loading¶

Fixture kullanımı

Fixture = veritabanına yüklenen hazır veri

```
class AnimalTestCase(TestCase):
    fixtures = ["mammals.json", "birds"]
```

Çalışma şekli

- setUpClass() içinde yüklenir
- TransactionTestCase:

    - Her test sonrası DB temizlenir
    - Fixture yeniden yüklenir

URLconf configuration¶

URL ayarı

```
@override_settings(ROOT_URLCONF=...)
```

Multi-database support¶

Çoklu veritabanı

```
databases = {"default", "other"}
```

- Hangi DB'lerin temizleneceğini belirler

Overriding settings¶

Ayar değiştirme

settings()

```
with self.settings(LOGIN_URL="/other/login/"):
```

modify_settings()

```
with self.modify_settings(MIDDLEWARE={"append": "..."}):
```

override_settings decorator

```
@override_settings(LOGIN_URL="/other/login/")
```

Isolating apps¶

App izolasyonu

```
@isolate_apps("app_label")
```

- Test modelleri için ayrı registry oluşturur

Email services¶

Email testleri

Testlerde email gönderilmez, outbox'a kaydedilir:

```
from django.core import mail

mail.outbox
```

Management Commands¶

Komut testleri

```
from django.core.management import call_command
```

Skipping tests¶

Test atlama

```
@skipIf
@skipUnless
```

Django özel skip

```
@skipIfDBFeature("supports_transactions")
@skipUnlessDBFeature("supports_transactions")
```

Assertions¶

Assertion'lar

Örnekler

- assertRaisesMessage
- assertWarnsMessage
- assertContains
- assertTemplateUsed
- assertRedirects
- assertHTMLEqual
- assertJSONEqual

Query testleri

```
self.assertNumQueries(2)
```

Tagging tests¶

Test etiketleme

```
@tag("fast")
```

Çalıştırma:

```
./manage.py test --tag=fast
```

Testing asynchronous code¶

Async testler

- async def test yazılmalı
- AsyncClient kullanılmalı

```
response = await self.async_client.get("/url/")
```

AsyncClient¶

- Normal client ile aynı ama async
- await gerekir

Warning

Decorator'lar async uyumlu olmalı:

```
@async_to_sync
```

modify_settings(*args, **kwargs)[source]¶

Benzer şekilde, Django `modify_settings()` decorator'ünü de sağlar:

```
from django.test import TestCase, modify_settings


class MiddlewareTestCase(TestCase):
    @modify_settings(
        MIDDLEWARE={
            "append":
"django.middleware.cache.FetchFromCacheMiddleware",
            "prepend":
"django.middleware.cache.UpdateCacheMiddleware",
        }
    )
    def test_cache_middleware(self):
        response = self.client.get("/")
        # ...
```

Decorator test sınıflarına da uygulanabilir:

```
from django.test import TestCase, modify_settings


@modify_settings(
    MIDDLEWARE={
        "append":
"django.middleware.cache.FetchFromCacheMiddleware",
        "prepend": "django.middleware.cache.UpdateCacheMiddleware",
    }
```

```
)
class MiddlewareTestCase(TestCase):
    def test_cache_middleware(self):
        response = self.client.get("/")
        # ...
```

Not

Bir sınıfa uygulandığında, bu decorator sınıfı doğrudan değiştirir ve yeni bir kopya oluşturmaz.
Bu nedenle farklı bir isimle atamaya çalışırsanız bile orijinal sınıf etkilenmeye devam eder.

Bir sınıf için:

- `modify_settings()` her zaman `override_settings()`'ten sonra uygulanır.

Uyarı

Settings dosyasındaki bazı ayarlar yalnızca Django başlatılırken okunur.

- `override_settings()` ile değiştirseniz bile
- Django'nun iç mekanizmaları bunu fark etmeyebilir

Öneriler:

- DATABASES ayarını değiştirmeyin
- CACHES ayarı dikkatli kullanılmalı
- settings değerlerini module-level constant olarak tanımlamayın

Bir ayarı tamamen kaldırma

```
@override_settings()
def test_something(self):
    del settings.LOGIN_URL
```

setting_changed sinyali

Django, ayar değiştiğinde bazı şeyleri sıfırlar:

Ayar	Sıfırlanan

USE_TZ, TIME_ZONE	DB timezone
TEMPLATES	Template engine
FORM_RENDERER	Renderer
SERIALIZATION_MODULES	Serializer cache
LOCALE_PATHS, LANGUAGE_CODE	Çeviriler
STATIC_ROOT, STATIC_URL, STORAGES	Storage

Isolating apps¶

Uygulama izolasyonu

```
from django.test.utils import isolate_apps
```

Amaç:

- Test sırasında oluşturulan modellerin çakışmasını önlemek

Kullanım (decorator)

```
@isolate_apps("app_label")
def test_model_definition(self):
    class TestModel(models.Model):
        pass
```

Context manager

```
with isolate_apps("app_label"):
    class TestModel(models.Model):
        pass
```

attr_name kullanımı

```
@isolate_apps("app_label", attr_name="apps")
class TestModelDefinition(SimpleTestCase):
    def test_model_definition(self):
        class TestModel(models.Model):
            pass
```

```
        self.assertIs(self.apps.get_model("app_label", "TestModel"),
TestModel)
```

kwarg_name kullanımı

```
class TestModelDefinition(SimpleTestCase):
    @isolate_apps("app_label", kwarg_name="apps")
    def test_model_definition(self, apps):
        class TestModel(models.Model):
            pass

        self.assertIs(apps.get_model("app_label", "TestModel"),
TestModel)
```

Emptying the test outbox¶

Test email kutusunu temizleme

Django testlerinde:

- Email gönderilmez
- `mail.outbox` içine kaydedilir

Her test başında otomatik temizlenir.

Manuel temizleme:

```
from django.core import mail

mail.outbox = []
```

Assertions¶

Doğrulamalar

Django, standart `assertEqual()` gibi fonksiyonlara ek olarak özel assertion'lar sağlar.

assertRaisesMessage

```
with self.assertRaisesMessage(ValueError, "invalid literal"):
    int("a")
```

- Belirli hata ve mesaj kontrolü yapar

assertWarnsMessage

- Warning kontrolü yapar (aynı mantık)

assertFieldOutput

Form alanlarını test eder:

```
self.assertFieldOutput(
    EmailField,
    {"a@a.com": "a@a.com"},
    {"aaa": ["Enter a valid email address."]}
)
```

assertFormError

```
self.assertFormError(form, "email", ["Hata mesajı"])
```

assertFormSetError

Formset hatalarını test eder

assertContains

```
self.assertContains(response, "text")
```

assertNotContains

Response içinde olmamasını kontrol eder

assertTemplateUsed

```
self.assertTemplateUsed(response, "index.html")
```

assertTemplateNotUsed

Template kullanılmadı mı kontrol eder

assertURLEqual

- Query param sırasını önemsemez

assertRedirects

```
self.assertRedirects(response, "/login/")
```

assertHTMLEqual

HTML karşılaştırır:

- boşlukları yok sayar
- attribute sırası önemli değildir

assertHTMLNotEqual

HTML eşit olmamalı

assertXMLEqual / assertXMLNotEqual

XML karşılaştırma

assertInHTML / assertNotInHTML

HTML içinde içerik arar

assertJSONEqual / assertJSONNotEqual

JSON karşılaştırma

TransactionTestCase özel assertion'lar

assertQuerySetEqual

```
self.assertQuerySetEqual(qs, ["a", "b"])
```

assertNumQueries

```
with self.assertNumQueries(2):
    ...
```

Tagging tests¶

Test etiketleme

```
from django.test import tag

@tag("fast")
def test_fast(self):
    pass
```

Çalıştırma

```
./manage.py test --tag=fast
```

Birden fazla tag

```
./manage.py test --tag=fast --tag=core
```

Hariç tutma

```
./manage.py test --exclude-tag=slow
```

Testing asynchronous code¶

Asenkron testler

- `async def` kullanılmalı
- Django otomatik event loop oluşturur

AsyncClient

```
from django.test import AsyncClient
```

Kullanım

```
async def test_my_thing(self):
    response = await self.async_client.get("/url/")
```

Farklar

- await gerekir
- ASGIRequest kullanır

Uyarı

Decorator'lar async uyumlu olmalı:

```
from asgiref.sync import async_to_sync

@async_to_sync
async def test():
    ...
```

Email services¶

Email test sistemi

Django test sırasında:

- gerçek email göndermez
- sahte outbox kullanır

Kullanım

```
from django.core import mail

self.assertEqual(len(mail.outbox), 1)
self.assertEqual(mail.outbox[0].subject, "Subject here")
```

Management Commands¶

Komut testleri

```
from django.core.management import call_command
```

Örnek

```
out = StringIO()
call_command("closepoll", poll_ids=[1], stdout=out)

self.assertIn("Successfully closed poll", out.getvalue())
```

Skipping tests¶

Test atlama

```
@skipIf
@skipUnless
```

Django özel

```
@skipIfDBFeature("supports_transactions")
@skipUnlessDBFeature("supports_transactions")
```

Mantık

- DB özelliği varsa → test atlanır
- DB özelliği yoksa → test çalışır